

투명 블럭 (Ghost Platform) — 구현 가이드라인

작성 대상: 외부 프로그래머
엔진: Godot 4.6.1 / 언어: GDScript
작성일: 2026-05-27

1. 개요 및 동작 규칙

핵심 컨셉

"색이 없으면 존재하지 않는다. 색을 입혀야 실체가 된다."

상태	플레이어 통과	밟기	사망 판정
투명 (미칠)	✔ 통과됨	✖ 불가	✖ 없음
검정 칠해짐	✖ 막힘	BLACK 플레이어만 ✔	WHITE 플레이어 💀
흰색 칠해짐	✖ 막힘	WHITE 플레이어만 ✔	BLACK 플레이어 💀

색 칠하는 방법

- 플레이어가 **총알을 발사**해 투명 블럭에 맞으면 칠해짐
- 이미 칠해진 상태에서 다른 색 총알을 쏘면 **색이 교체**됨
- 한 번 칠해지면 **투명 상태로 되돌아가지 않음** (기획에 따라 변경 가능)

2. 물리 레이어 설정

project.godot에 레이어 추가 필요

```
[layer_names]
2d_physics/layer_1 = "player"
2d_physics/layer_2 = "platform_black"
2d_physics/layer_3 = "platform_white"
2d_physics/layer_4 = "platform_gray"
2d_physics/layer_5 = "bullet"
2d_physics/layer_6 = "paint_detector"
2d_physics/layer_7 = "kill_zone"
2d_physics/layer_8 = "ghost_interact"    ← 신규 추가
```

레이어 8 (ghost_interact) 역할

- 투명 블럭이 **항상** 이 레이어에 존재
- 플레이어 collision_mask에는 포함 안 됨 → **플레이어는 통과**
- 총알 collision_mask에 추가 → **총알은 항상 감지 가능**

3. 씬 구조 (GhostPlatform.tscn)

```

GhostPlatform (StaticBody2D) ← ghost_platform.gd
├─ CollisionShape2D          ← 물리 충돌 영역 (layer 상태에 따라 변경)
├─ DeathDetector (Area2D)    ← layer=64(kill_zone), 칠해졌을 때만 monitorable=true
│   └─ CollisionShape2D      ← StaticBody2D와 동일 형태 권장
├─ SpriteUnpainted (Sprite2D) ← 투명 상태 텍스처 (아지랑이 이미지)
└─ SpritePainted (Sprite2D)  ← 칠해진 상태 텍스처 (검정 or 흰색)

```

Sprite를 2개로 나누는 이유

투명 상태와 칠해진 상태의 텍스처가 완전히 다르기 때문.
상태 전환 시 **visible** 토글로 간단하게 교체 가능.

4. Collision Layer & Mask 설정값

StaticBody2D (GhostPlatform 본체)

상태	collision_layer	collision_mask
투명 (미칠)	128 (layer8만)	0
검정 칠해짐	128 2 = 130	0
흰색 칠해짐	128 4 = 132	0

비트 계산

layer8 = 1 << 7 = 128

LAYER_BLACK = 1 << 1 = 2

LAYER_WHITE = 1 << 2 = 4

DeathDetector (Area2D)

상태	collision_layer	monitorable
투명 (미칠)	64	false
검정 칠해짐	64	true
흰색 칠해짐	64	true

⚠ monitorable 반드시 false로 시작

투명 상태에서 ColorSensor에 감지되면 안 됨

총알 (bullet.gd) — 기존 코드 수정 필요

기존

mask = 14 # layer2(black) + layer3(white) + layer4(gray)

```
# 수정
mask = 142 # 14 + 128(ghost_interact)
```

⚠ **bullet.tscn의 collision_mask를 142로 변경할 것**

5. ghost_platform.gd 구현 가이드

상수 정의

```
const LAYER_GHOST: int = 1 << 7 # 128
const LAYER_BLACK: int = 1 << 1 # 2
const LAYER_WHITE: int = 1 << 2 # 4
const LAYER_DEATH: int = 1 << 6 # 64
```

상태값

```
# -1: 미칠 상태 (투명)
# 0: 검정 칠해짐 (ColorDefs.BLACK)
# 1: 흰색 칠해짐 (ColorDefs.WHITE)
var paint_color: int = -1
```

_ready() 초기 설정

```
func _ready() -> void:
    add_to_group("paint_bodies") # 총알이 update_color() 호출 가능하게
    collision_layer = LAYER_GHOST # 투명 상태 초기값
    $DeathDetector.monitorable = false # 판정 비활성화
    $SpritePainted.visible = false # 칠해진 스프라이트 숨김
    $SpriteUnpainted.visible = true # 투명 스프라이트 표시
```

update_color() — 총알 명중 시 호출됨

```
func update_color(new_color: int) -> void:
    paint_color = new_color
    _apply_color()

func _apply_color() -> void:
    match paint_color:
        ColorDefs.BLACK:
            collision_layer = LAYER_GHOST | LAYER_BLACK
        ColorDefs.WHITE:
            collision_layer = LAYER_GHOST | LAYER_WHITE
```

```
# DeathDetector 활성화
$DeathDetector.monitorable = true

# 비주얼 전환
$SpriteUnpainted.visible = false
$SpritePainted.visible = true
$SpritePainted.modulate = Color.BLACK if paint_color == ColorDefs.BLACK \
    else Color.WHITE
```

에디터 힌트 (@tool)

```
@tool
extends StaticBody2D
```

에디터에서 실시간으로 상태 확인이 필요하다면 추가.

단, @tool 사용 시 `_ready()` 내부에 `if Engine.is_editor_hint(): return` 추가 필수.

6. 사망 판정 동작 방식

기존 플랫폼과 동일한 구조 사용

투명 블럭이 칠해지면 일반 플랫폼과 **완전히 동일한 판정 흐름**이 적용됨.

```
ColorSensor(플레이어)가 DeathDetector를 감지
├─ death_zones 그룹에서 부모(GhostPlatform) 가져옴
│   └─ paint_color != player_color → die()
│       paint_color == player_color → 안전
```

⚠ DeathDetector를 death_zones 그룹에 추가해야 함

```
# _ready() 내부
$DeathDetector.add_to_group("death_zones")
```

기존 Platform.tscn의 DeathDetector와 동일한 그룹명 사용.

ColorSensor가 감지 후 `plat.get("paint_color")`로 색 조회함.

⚠ color_state vs paint_color 네이밍 주의

기존 Platform은 `color_state` 프로퍼티를 사용함.

GhostPlatform은 `paint_color` 사용 예정.

player.gd의 `_check_color_death()`가 `get("color_state")`로 값을 읽는다면

GhostPlatform도 동일하게 `color_state` 프로퍼티명을 맞춰야 함.

→ **player.gd 코드 확인 후 프로퍼티명 통일할 것**

7. 페인트 덮어쓰기 동작

총알이 이미 칠해진 GhostPlatform에 닿으면:

```
bullet._on_body_entered(body)
└─ body.is_in_group("paint_bodies") → true
    └─ body.call_deferred("update_color", bullet_color)
        └─ GhostPlatform.update_color(새 색) 호출
            → collision_layer 교체
            → 비주얼 교체
```

기존 PaintMark와 동일한 흐름. 별도 처리 불필요.

8. 주의사항 & 엣지 케이스

⚠ 플레이어는 블럭 내부에 있을 때 칠해지는 경우

플레이어가 투명 블럭을 통과하는 도중 총알이 맞으면
블럭이 갑자기 solid가 되어 **플레이어가 블럭 안에 끼일 수 있음.**

대응 방안 (택1):

- 칠해지는 순간 블럭 내부에 플레이어가 있는지 체크 후 튕겨내기
- 칠해질 때 0.1~0.2초 딜레이 후 collision_layer 적용 (플레이어가 빠져나갈 시간)
- 블럭 크기를 플레이어보다 작게 설계 (기획 단계에서 방지)

⚠ call_deferred 사용 위치

총알의 **body_entered** 콜백 내에서 **update_color**가 호출됨.
update_color 내부에서 **collision_layer** 변경은 물리 flush 중 발생할 수 있음.

```
# 안전한 방법
func update_color(new_color: int) -> void:
    paint_color = new_color
    call_deferred("_apply_color") # collision_layer 변경을 다음 프레임으로 미룸
```

⚠ 투명 상태에서 DeathDetector monitorable = false 필수

투명 상태에서 monitorable이 true면
플레이어가 블럭을 통과할 때 ColorSensor가 DeathDetector를 감지해
아무 이유 없이 사망하는 버그 발생.

⚠ 총알 mask 수정 안 하면 총알이 투명 블럭 통과

bullet.tscn의 collision_mask에 layer8(128) 추가 필수.

안 하면 투명 블럭에 총알이 닿지 않아 칠하기 불가.

9. 구현 체크리스트

- project.godot에 layer_8 = "ghost_interact" 추가
- bullet.tscn collision_mask = 142 로 수정
- GhostPlatform.tscn 씬 생성
 - StaticBody2D + ghost_platform.gd
 - CollisionShape2D 형태 설정
 - DeathDetector (Area2D) 추가 – layer=64, monitorable=false로 시작
 - SpriteUnpainted (투명 텍스처)
 - SpritePainted (칠해진 텍스처)
- ghost_platform.gd 작성
 - add_to_group("paint_bodies") – _ready()
 - add_to_group("death_zones") – DeathDetector에 적용
 - update_color() 구현
 - _apply_color() 구현 (collision_layer, monitorable, 비주얼 전환)
- player.gd의 _check_color_death()에서 color_state 읽는 방식 확인
 - GhostPlatform의 프로퍼티명과 일치시킬 것
- 엣지 케이스 테스트
 - 통과 중 칠해질 때 끼임 현상 확인
 - 칠한 후 사망 판정 정상 동작 확인
 - 색 교체 후 판정 정상 동작 확인

10. 연관 파일 목록

파일	수정 여부	내용
project.godot	✓ 수정	layer_8 추가
scenes/bullet/Bullet.tscn	✓ 수정	collision_mask 142로 변경
scripts/ghost_platform.gd	✓ 신규	투명 블럭 로직
scenes/platforms/GhostPlatform.tscn	✓ 신규	투명 블럭 씬
scripts/player.gd	⚠ 확인 필요	color_state 프로퍼티명 확인
scripts/bullet.gd	✗ 수정 불필요	기존 로직 그대로 사용 가능

이 문서는 기획/개발 협업용 가이드라인입니다. 구현 중 변경사항은 문서에 반영해주세요.